Dynamic Initialization of @Bean using @Value annotation

I told you about Unsatisfied Dependency problem and how to resolve it using @Qualifier annotation.

There are two solution :

1st Solution :- ~~Dynamic~~ this is hardcoded ; we can write the name with @Qualifier

```
@RestController
@RequestMapping(value = "/api")
Public class User {

    @Qualifier("onlineOrderObject")
    @Autowired
    Order onlineOrderObject;

    @Qualifier("offlineOrderObject")
    @Autowired
    Order offlineOrderObject;

    @PostMapping("/createOrder")
    Public ResponseEntity<String> CreateOrder(@RequestParam boolen
                                                isOlineOrder)
        if (isOnlineOrder) {
            onlineOrder.createOrder();
        }else {
            offlineOrder.createOrder();
        }

        return ResponseEntity.ok("");
    }
}
```

Concept && Coding

Dynamic : we can make another External Configuration class. ②

```
@Configuration
Public class AppConfig{

  @Bean
  Public Order createOrderBean(@Value("${isOnlineOrder}")
                                boolean isOnlineOrder){

    if( isOnlineOrder){
      return new OnlineOrder();
    }else {
      return new ~~Online~~
                 Offline Order();
    }
  }
}
```

application.propert
isOnlineOrder=false

@Value : It is used to inject values from various sources like property file, environment variables or inline literals.

```
@Value("false")
```

Concept && Coding